

Relações de Recorrência

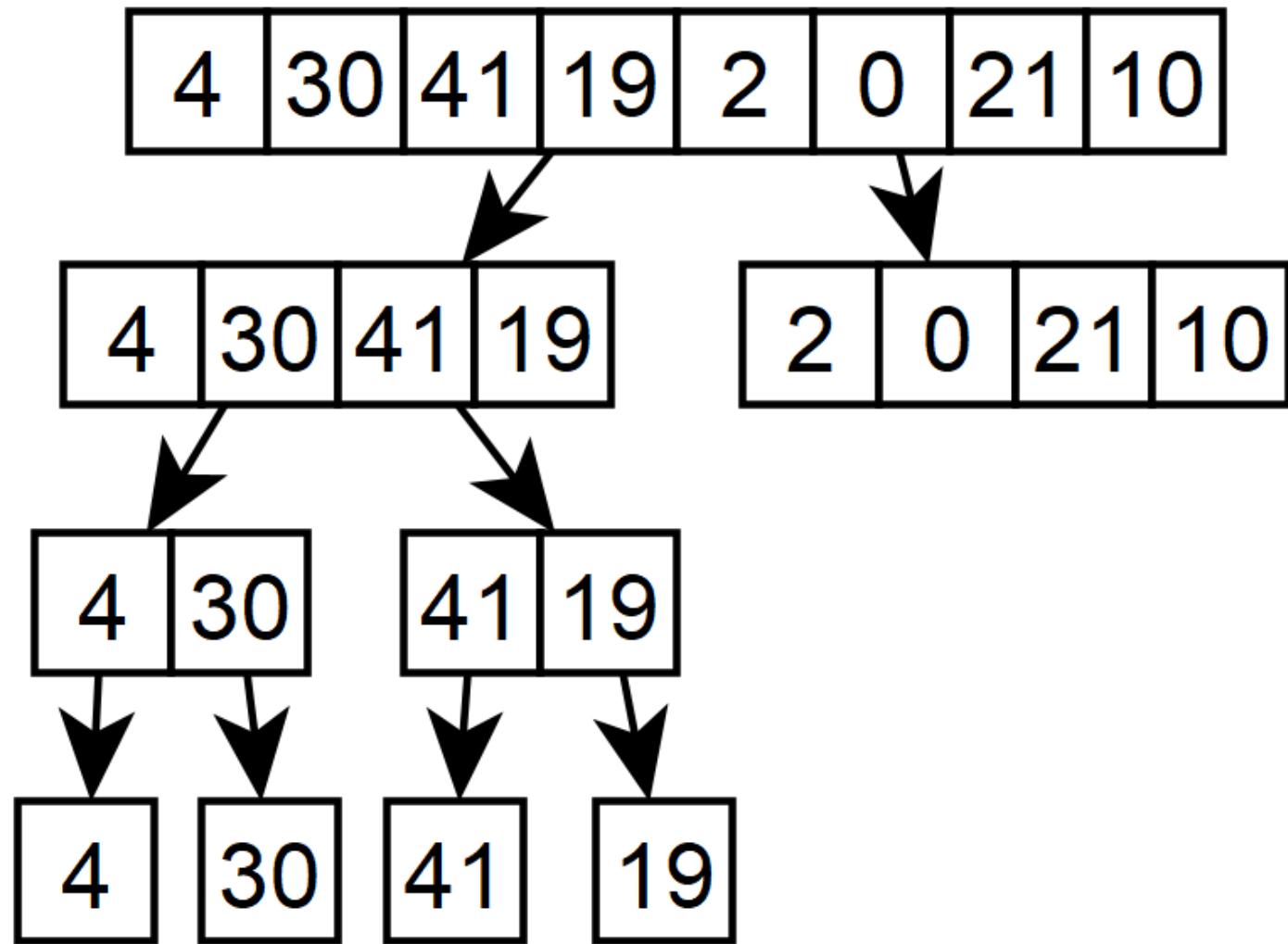
Merge-Sort

Prof. Dr. Leandro I. Pinto

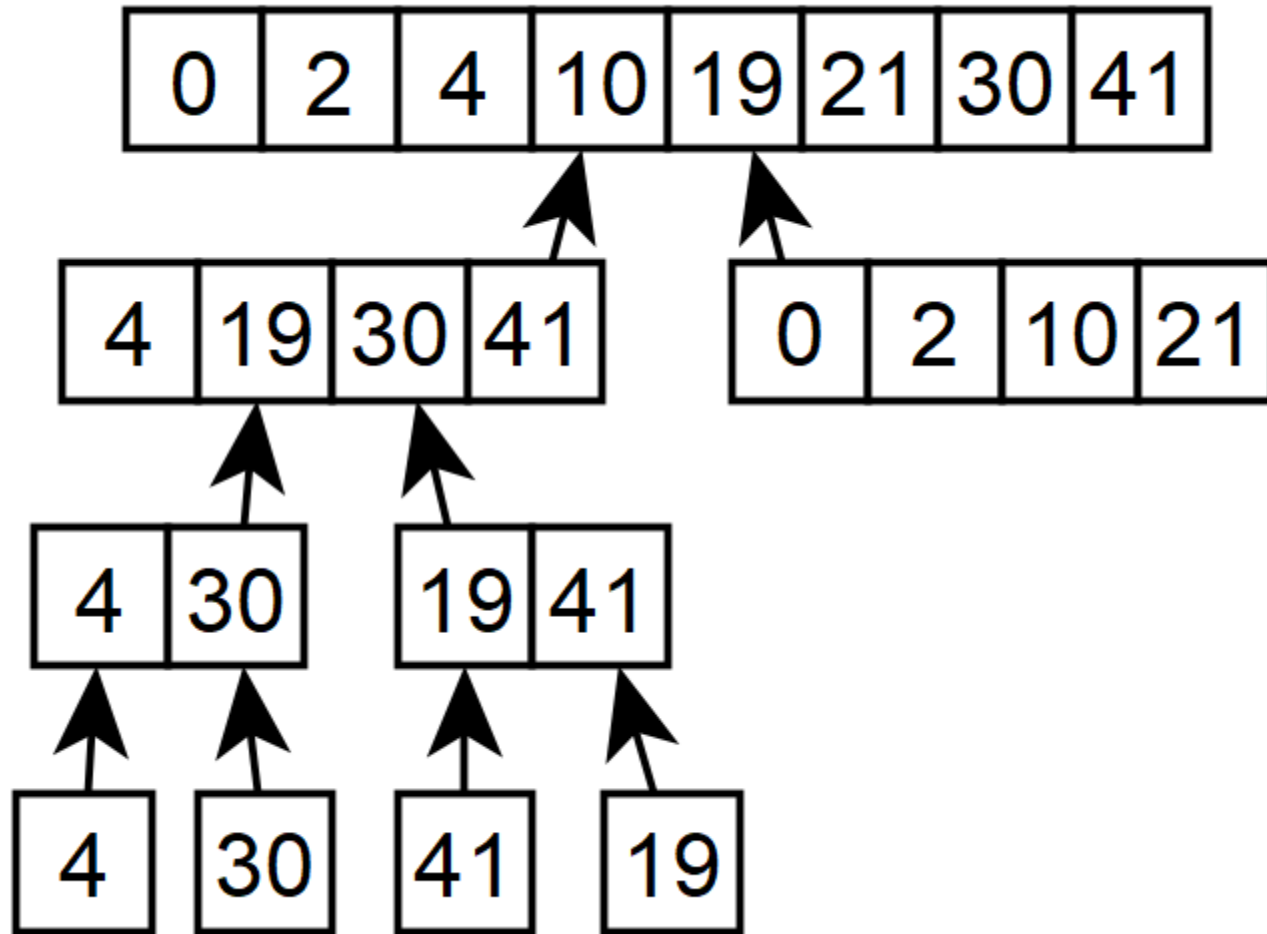
Merge-Sort

- Algoritmo de ordenação que utiliza uma abordagem de divisão e conquista;
- Dada uma entrada, ele a divide ao meio sucessivamente;
- Cada chamada recursiva recebe metade do vetor para ordenar;
- Uma função de MERGE une as metades;

Merge-Sort 1



Merge-Sort 2



Análise Merge Sort

```
10 void merge(int *v, int p, int q, int r){
11     int *out, i, i1, i2;
12     int n = r - p + 1;
13     |
14     out = (int*)malloc(sizeof(int)*n);
15     i1 = p; i2 = q;
16     i = 0;
17
18     while(i1 < q && i2 <= r){
19         if(v[i1] < v[i2]){
20             out[i] = v[i1];
21             i1++;
22         }else{
23             out[i] = v[i2];
24             i2++;
25         }
26         i++;
27     }
28
29     while(i1 < q){
30         out[i] = v[i1];
31         i++;
32         i1++;
33     }
34     while(i2 <= r){
35         out[i] = v[i2];
36         i++;
37         i2++;
38     }
39
40     for(i=0; i<=(r-p); i++){
41         v[p+i] = out[i];
42     }
43
44     free(out);
45 }
```

Análise Merge Sort

- $O(n)$

```
10 void merge(int *v, int p, int q, int r){
11     int *out, i, i1, i2;
12     int n = r - p + 1;
13     |
14     out = (int*)malloc(sizeof(int)*n);
15     i1 = p; i2 = q;
16     i = 0;
17
18     while(i1 < q && i2 <= r){
19         if(v[i1] < v[i2]){
20             out[i] = v[i1];
21             i1++;
22         }else{
23             out[i] = v[i2];
24             i2++;
25         }
26         i++;
27     }
28
29     while(i1 < q){
30         out[i] = v[i1];
31         i++;
32         i1++;
33     }
34     while(i2 <= r){
35         out[i] = v[i2];
36         i++;
37         i2++;
38     }
39
40     for(i=0; i<=(r-p); i++){
41         v[p+i] = out[i];
42     }
43
44     free(out);
45 }
```

Análise Merge Sort

```
47 void mergeSort(int *v, int p, int r){  
48     int q;  
49     printArray(&v[p], (r-p)+1);  
50     if(p<r){  
51         q = (p+r)/2;  
52         mergeSort(v,p,q);  
53         mergeSort(v,q+1,r);  
54         merge(v,p,q+1,r);  
55     }  
56 }
```

Análise Merge Sort

```
47 void mergeSort(int *v, int p, int r){
48     int q;
49     printArray(&v[p], (r-p)+1);
50     if(p<r){
51         q = (p+r)/2;
52         mergeSort(v,p,q);
53         mergeSort(v,q+1,r);
54         merge(v,p,q+1,r);
55     }
56 }
```

- $T(n) = 2T\left(\frac{n}{2}\right) + n$

- $T(1) = 1$

- $T(8) = 2T(4) + 8$

- $2T(4) = 2 \cdot 2T(2) + 2 \cdot 4$

- $2^2T(2) = 2^3T(1) + 2^3$

- $T(8) = 8 + 8 + 8 + 8$

Análise Merge Sort

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

$$2T(n) = 2^2T\left(\frac{n}{2^2}\right) + 2\frac{n}{2}$$

$$2^2T\left(\frac{n}{2^2}\right) = 2^3T\left(\frac{n}{2^3}\right) + 2^2\frac{n}{2^2}$$

$$2^{l-1}T\left(\frac{n}{2^{l-1}}\right) = 2^lT\left(\frac{n}{2^l}\right) + 2^{l-1}\frac{n}{2^{l-1}}$$

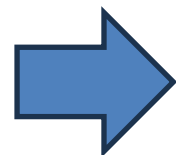
$$\frac{n}{2^l} = 1$$

$$n = 2^l$$

$$l = \log_2 n$$

$$n + n + n + \dots + n + 2^lT\left(\frac{n}{2^l}\right)$$

$$n \log_2 n + n1$$



$$O(n \log n)$$